

Taxonomy of Malware Analysis in the IoT

Silvia Wahballa Soliman
Misr International university
silvia.wahballa@miuegypt.edu.eg

Mohammed Ali Sobh
Ain Shams University
mohamed.sobh@eng.asu.edu.eg

Ayman M. Bahaa-Eldin
Misr International University
On leave from Ain Shams University
ayman.bahaa@eng.asu.edu.eg

Abstract—Internet of Things (IoT) totally transformed our world on many levels and in many fields either industry, health, economy, energy saving, etc. According to Cisco by 2020 there will be about 50 billion IoT connected devices and this causes many issues specially security problems. IoT devices are widely used everywhere and leads to user privacy, big data as well as in data and server management problems but malware is from the biggest challenges facing IoT devices where there are different types of malware like viruses, Trojan horses, worms, spy-ware and viruses, which can access and manipulate confidential information or cause a running problem in the system. As there are many types of malware but at the same time there are different approaches for malware analysis. Therefore there are many approaches to handle malware analysis and each approach has its pros and cons. In this paper we will discuss security issues in IoT devices and different approaches for malware analysis and build a taxonomy for malware analysis tools and methodologies from different points of view as well as the advantages and disadvantages of each approach.

Keywords—dynamic analysis, malware, static analysis, Internet of things, security.

I. INTRODUCTION

Internet of Things (IoT) field now is becoming very wide and IoT connections will reach 46% by 2020 [16], which means that more security issues will arise as IoT applications extend to very sensitive areas like financial services, manufacturing, etc. Therefore many issues are raised due to the vast usage of IoT devices either security, privacy, management, inter-operability issues [11]. Thus we are concerned about securing the IoT devices and securing the software to remove software vulnerability and detecting any unwanted or unexpected behavior that can be caused by many things like malware.

Malware (stands for malicious software) is any software that can cause harm to the device or does any unwanted action on a computer system. Most problems caused nowadays in software are caused by malware as it can cause harm to the system or the network and can be installed on the system without the knowledge of its owner. Malware analysis is the art of dissecting the malware to be able to understand its effect and detect its behavior and know how to avoid its effect [17]. There are many malware analysis tools that detect the malware type and the harms it will cause. Each tool or algorithm has its pros and cons and the malware analysis tool is chosen based on the scope in which the analysis is done.

The problem in IoT devices is restricted computational, memory and energy resources and malware analysis tools are resource exhausting, so applying malware analysis tools on

IoT devices will be impractical.

In this paper we are going to discuss IoT security challenges and the problem of malware in IoT as well as different methodologies and technologies of malware analysis and build a taxonomy for these approaches and mention the pros and cons of each one.

The first section will mention the related work to the scope of malware analysis tools in the IoT devices that were recently introduced, then section two will discuss the IoT security issues and challenges, section three will talk about different malware analysis approaches and methodologies in general then the fourth section that discusses the investment of some malware analysis approaches in the recent research trials in the IoT field and finally the conclusion and future work section.

II. RELATED WORK

Malware analysis is an important scope that is currently under research, Cloud Eyes in [25] is a cloud based malware detection platform is introduced and it is mainly dedicated for resource constrained IoT devices as it is a lightweight scanning system that protects data privacy and reduces the cost of communication, it detects malware based on signature detection mechanism built on reversible sketch structure and introduces suspicious bucket cross filtering, it provides the security service with high performance. Also [2] proposes an embedded security framework for IoT, which introduces AES-GCM-based security protocol that is used for encryption and authentication, also in this paper threshold based jamming countermeasure defensive mechanism against jamming attacks is proposed as well as a key management algorithm to manage keys under Wireless Sensor Networks, also in this paper game theory based countermeasure to detect jamming attacks by cross layer features is proposed.

In [24] a system that proposes an inHub security manager for vulnerable IoT devices, which intercepts all in and out traffic to and from smart home devices to increase security for these devices. In [20] a secure compiler for developing secure IoT services is proposed as secure software is needed in IoT devices to avoid software vulnerabilities and this is done by using a compiler and virtual machine that use secure coding rules and checking tools.

III. INTERNET OF THINGS SECURITY ISSUES

Internet of Things is currently invading our world and after three years from now billions of devices will be connected together. IoT devices now are used in many scopes like business, transportation, education, energy saving, home, industry, health, etc. Therefore many challenges arise due to the complex and heterogeneous model of the IoT. Therefore as [11] stated there are inter-operability, security, privacy, integration and management challenges. Internet of Things is built from many components [2] as shown in Fig. 1 like communication between devices, data processing, identification of devices, device localization and data storage, all of these are based on network connection, sensors and IoT devices. The figure shows that without security the whole IoT structure can fall apart.

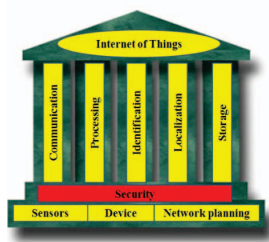


Fig. 1. IoT pillars [2].

In [23] that talks about Industrial IoT devices, which consists of electronics that are driven by software, electronics are prone to physical attack like hardware attacks and side channel attacks and software is prone to malicious code attacks like Trojan horses, viruses, etc also communication that is prone to attacks like Man in the Middle (MiM) and Denial of service (DoS).

Thus the problem is that attacks on the IoT devices nowadays are on various levels [2]:

- Physical attacks that are concerned with the hardware components and this is harder to implement, from its methods is de-packaging of chip.
- Side channel attacks that retrieve information from encryption device other than plain-text nor cipher-text but can be timing analysis attacks, power analysis attacks, fault analysis attacks to make use of this data and try to get the key used in encryption-decryption process.
- Crypt-analysis attacks Focus on the cipher-text and try to get the plain-text, this type of attacks is like Man In the Middle, known plain-text, known cipher-text, etc.
- Software attacks It is the main source of vulnerabilities in IoT devices, from its types is jamming attacks, worms, Trojan horses, logic bombs, viruses that can cause problems like buffer overflow.
- Network attacks Since IoT devices are connected together through wireless communication that is more vulnerable to attacks like eavesdropping, Replication attacks, routing attacks, traffic analysis, message corruption, distributed denial of service attacks (DDoS).

Then security is a great challenge to the IoT devices such as Radio Frequency Identification (RFIDs), household appliances, embedded devices, sensor nodes, cell phones because of many issues as shown in Fig. 2 [11] .

According to [11] the major security issues are:

A. end to end security

It is protecting end to end communication between two IoT devices without being read or edited and this is a serious challenge in the heterogeneous IoT architecture. In addition IoT devices cannot run protocols like Transport Layer Security (TLS) [8] -which provide secure communication over the internet to avoid forgery and eavesdropping - or Internet Protocol Security (IPSec) protocol -that authenticates and encrypts packets of data sent- due to low computational, power resources.

B. Data security

Protection of personal and critical data from external attacks or from malicious behavior of software on IoT devices where accessing this data may cause catastrophic results.

C. Identity and access management

Protection from attacks where the hacker acts as another identity other than the real authorized person, this is called masquerading

D. Physical security

To secure the IoT device against Distributed Denial of Service (DDoS) attacks where the targeted device is flooded with requests till its resources is exhausted, then an intrusion detection system is needed but the implementation of such systems in IoT is currently under research.

E. Compliance

IoT usage should comply to the data protection and privacy laws that includes user consent in which the user should agree on any access to his data, freedom of choice in which the user should have a total freedom to choose being involved in a communication or not, anonymity where user has the right to stay anonymous if the service need no identity.

F. governance and privacy

Privacy is a major challenge in IoT devices as personal information became more vulnerable to attackers that want to access information about the environment and the user. Where IoT devices in smart home, smart cars log data about the environment and the users so this breaches user privacy.

G. Authorization, Authentication and access control

Role Based Access Control (RBAC) is a model that assigns permissions to users according to the available permissions versus the users in the system but the issue in RBAC is that in IoT there is huge number of devices so assigning a role to each device in the network is a very complex task and this is called "role explosion" therefore instead Attribute

Based Access Control (ABAC) model is used where access is granted to the user based on the attributes he possesses, attribute can be user, resource, object, etc. The advantage of ABAC over RBAC is being dynamic, decisions are policy driven, more flexible, minimize data access.

There are many security requirements mentioned in [2][11][23] about IoT devices like authorization that prevent unauthorized access to the IoT device, authentication, which checks the identity of the one who is requesting a certain service which can be done using Secure Socket Layer (SSL) protocol but it's not practical in IoT devices, which have constrained resources, confidentiality in which sensitive data should not be accessible to anyone, Integrity where data should not be modified, secure network access like protecting the network of the IoT system from any malware or vulnerabilities, secure software execution that protects the IoT device against software weaknesses like trying to overwrite stack contents using certain function call, availability that means that data should always be available and access to it should not be prevented by unauthorized users.

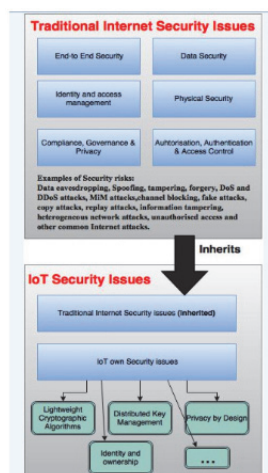


Fig. 2. Security issues in IoT[11].

IV. TAXONOMY FOR MALWARE ANALYSIS APPROACHES

A. MODE OF ANALYSIS

We can classify malware analysis methods by the mode of analysis whether it is static, dynamic or a mix from both (hybrid). The difference between static and dynamic analysis is shown in Table I.

1) *Static Analysis*: Static analysis is analyzing the software without executing it, it looks at the file itself and tries to extract information about the structure and the data in the file such that the time the program is compiled, which compiler is used, information about structure and data in the file can be determined. This can be done either on the source code or the binary executable. The issue is that when the code is compiled from source code to binary code some information will be lost and the analysis of the code will be very complicated. While the good point here is that static analysis can identify specific coding errors that can lead to problems at run-time like crashes or memory-leaks. Static analysis can be classified into either basic or advanced static analysis. [17]

- 1) Basic static analysis Examines only the executable file without viewing the actual instructions, it's fast and simple but it may miss an advanced malicious behavior so it may be sometimes ineffective.
- 2) Advanced static analysis Examines the program instructions to discover what the program does exactly but it needs advanced knowledge about dis-assembly and operating systems concepts.

From the Techniques supported in static analysis [13] file fingerprinting, extraction of hard coded strings, anti-virus scanning, packer detection, dis-assembly, Signature based and Heuristic (behavior) based.

- File fingerprinting is done using hashing (cryptographic hash) to check whether the file has been modified or not.
 - Extraction of hard coded strings examine the embedded strings, which are in the form of status or error messages.
 - Antivirus scanners are used to prevent, detect and remove malwares.
 - Packer detection most malwares nowadays are distributed in obfuscated form (obfuscation refers to transformation of the binary such that parsing of the instructions becomes difficult) and this is done using packing, which makes the program look different and analysis can't detect malware in this case and can't detect its metadata, so if packing is detected then the analyzer suspects the presence of malicious software.
 - Dis-assembly is the reconstruction of assembly code from machine code and based on the reconstruction of the assembly code the malware analyst can inspect the program logic.
 - Signature based [6][27] is known as finger printing technique. It compares the signature of suspicious software to a list of significant malwares to discover if there is any malicious piece of code. The signature can either be hash signature, byte signature. But in case the malware authors use encrypted code of polymorphic behavior -polymorphic is when part of the code remains constant with each iteration-then we need something other than signature to detect the malware presence, therefore we need heuristic based analysis.
 - Heuristic (behavior) based known as pro-active technique. It Checks for signs of malware and compare it to a list of known malware behaviors whereas it checks execution time real path. It learns how a program executes to produce uncommon activities like creating specific registry keys, killing process or unpacking code.
- Heuristic based static analysis can be one of the following either File based where file data like contents, purpose, and destination are checked and detected if it contains any malicious behavior, also analysis can be Weight based where applications weight is checked if it exceeds the threshold of the danger it may possess and it can be Rule based where rules are extracted

from the application are checked whether they match or mismatch with the previously defined rules.

From the advantages of static analysis is that it can detect all execution paths also it is safe because the sample isn't really executed on the machine but it has some drawbacks because it's time consuming also the problem is that the source code of the malware is not always available so it cannot be easily analyzed also if the malware employs self-modifying (packing) technique or rely on values that can't be statically determined (current system date, indirect jump instructions) static analysis techniques becomes ineffective.

2) *Dynamic analysis*: Dynamic analysis is testing the program by executing it at real time and trying to find errors in the program while running, there are many ways to dynamically analyze a suspicious software as described in the following sections.

a) *Sandboxing*: Sandboxing [18] is a security mechanism, which separates running programs that are from an untrusted source where the sandbox offers a restricted set of resources as space on disk, memory and network access. A good sandbox has to achieve three goals: visibility that is to see as much as possible of the activity of the program, resistance to detection as the malware can change its behavior to evade detection as well as scalability to be able to analyze many samples in an automated fashion and with no interference.

In user mode the sandbox looks at the system call interface or the windows API. System calls can be like reading from files, sending packets over network, reading registry entry on windows, but the issue here is that the sandbox doesn't detect how malware processes the data it has just read so sandboxes need also to monitor the instructions executed. From examples of sandboxing is cuckoo sandbox. To be able to build a sandbox we can do this either using virtualization or emulation

The advantage of sandboxing is that it offers controlled and safe environment to run and test the malicious software but the problem or the drawback here is the lack of internet connection that is sometimes needed in some malware types due to show the real purposes and action of possible malware.

b) *Debugging*: Debugging [10][17] is considered type of advanced dynamic analysis. It gives you an insight to what the program is doing.

In debugging break points [7] are inserted, which are signals that suspends the program for a while at a certain line of code to detect whether the program functions as expected or not. Therefore breakpoints are used to hook into invocations of certain functions, hooking is then used to monitor system actions and memory content, register values and CPU state of the debugged process.

Debugging can be categorized as one of the following

- Source level vs assembly level debugging
- Kernel mode vs user mode debugging

At source level debugging that is most common, developer can debug while coding and usually done on the Integrated Development Environment (IDE) like visual studio or net beans. Assembly level debugging can be called low level debugging as it is done on the assembly level, which allows the developer to stop at each CPU instruction and check the memory locations.

In Kernel mode, there are no restrictions on the access to the device hardware to execute any CPU instruction or access any memory address. In User mode, the code has no direct access on hardware or memory but it needs system calls to switch from user mode to kernel mode to access hardware or memory. Kernel debugging is performed on two systems because there is only one kernel; if the kernel is at a break point, no applications can be running on the system. One system runs the code that is being debugged and another runs the debugger.

There are two popular examples for debuggers ollydbg that is used in user mode debugging and windbg that is for kernel mode debugging only. In a debugger there are two ways to debug a program one is to start the program with the debugger or attach the debugger to a program that is already running. Debugging sets breakpoints to be able to stop when a particular instruction is executed the control transfers to the debugger this is called software execution breakpoints, on the other side hardware execution breakpoints is done using dedicated hardware registers where every time the processor executes an instruction, there is hardware to detect if the instruction pointer is equal to the breakpoint address.

Debugging has many advantages as we can internally the program while running and the called functions and the arguments values can be tracked as well as it shows the current state of registers and memory during execution which gives a very good insight of the suspicious software, while the debugging has a disadvantage that it is not real time.

c) *Process monitoring*: Shows real time file system, registry, process and thread activity, here many information about processes is captured like input and output parameters of operations, thread stacks for each operation, process details (image path, command line, user and session ID), process tree, registry activity and this gathered information is used to detect if there is any suspicious behavior in the software being analyzed. One of the most important tools that does process monitoring is procMon tool.

d) *Binary diffing*: It's a tool used by malware analysts to find the difference between two binaries of certain suspicious software (in which bytes are changed and their corresponding offsets), this is important to analyze security updates, infected binaries, firmware changes, etc to detect if there is any problem or defect in the detected software. This is done by taking an image from system state before malware execution and another after malware execution and compare the two results. This method is easier to implement but only gains an overview of the system but with no account to the dynamic changes [30].

e) *Network monitoring*: From methods of dynamic analysis is network monitoring to trace the transactions going through the network

- 1) Netcat [15][17] :
This is a computer networking utility. Its called Swiss army knife because it has many features like port scanning, port forwarding, tunneling, proxying across TCP and UDP connections. It is used for debugging and investigation. It can be used in client mode for connecting or server mode for listening since most of malware communicates on port 80 (HTTP) and 443 (HTTPS) because they are the only ports not blocked by firewall
- 2) Wireshark [14][17] :
This is a network protocol analyzer and a packet sniffing tool, it helps the user to investigate whats going on in the network on a very deep level as it shows the live network traffic. It shows the protocols and the packet information and the size of the packet and many other useful information.

f) *Network services faking*: Network services can be faked using dynamic analysis tools to be able to protect the local machine from any untrusted requests or responses

- 1) ApateDNS[12][17] : DNS (Domain Name Server) is a service that is responsible of translation domain name to the corresponding IP address to locate and identify computer devices. ApateDNS is a tool that controls DNS responses and acts as a DNS server on your device. It spoofs DNS responses to the user specified IP address by listening on UDP port 53 which is used for domain name resolution.
- 2) INET Sim [29] : It is a linux based software suite used to simulate common internet services like HTTP (Hyper Text Transfer Protocol), HTTPS (Hyper Text Transfer Protocol Secured, DNS, FTP (File Transfer Protocol), like when the malware requests a file INetSim returns the file.

g) *Registry snapshots*: The registry [17] contains information about programs and hardware installed on your operating system, thus its important to track if the malware caused any changes in the registry values. In this method two registry methods are compared, you just take the first shot then run the malware and take the second snap shot then compare both together and checks the changes that occurred in the windows registry values (windows, program, driver) values stored in .ini file to detect if there is any unwanted behavior in the software being analyzed.

In dynamic analysis there are many advantages like being fast as we can detect problems caused by malware at run time, also we can analyze applications that we dont have code for and in dynamic analysis we validate or not the static analysis output.

On the contrary from the disadvantages of dynamic analysis that it sometimes produce false negatives and false positives and the performance of the dynamic analysis tool depends

only on the rules used in scanning as well as it is sometimes hard to track vulnerability back to its exact location.

TABLE I. COMPARISON BETWEEN DYNAMIC AND STATIC ANALYSIS METHODS.

Factors	Static Analysis	Dynamic Analysis
Time	Less time if automated but more time if conducted manually	More time is needed
Input	Source code, Byte code of interpreted language or binary code of a compiled application.	Memory snapshots and run-time data
Resource consumption	More cost efficient	Needs more resources in memory and processing
Accuracy	Less than dynamic analysis	Better because it detects run-time vulnerability
Advantages	Faster and code weaknesses are found earlier in the development life cycle, also in static analysis exact location of weakness is found, also it is more cost efficient than dynamic analysis. Static analysis analyzes the source code so it checks all possible malware executions.	Find vulnerabilities at run-time, more flexible, more accurate, helps in analysis of applications in which you do not have access to the source code. It is more attractive than static analysis because it is concerned with actual code execution.
Limitations	Cannot find vulnerabilities at run time. Also static analysis is sometimes hard to perform because the source code of malware is not always available. In addition the problem in static analysis is that sometimes code obfuscation techniques like compression, encryption or self-modification can be used to avoid analysis.	Analyzes only a single malware at a time.

B. TECHNIQUES SUPPORTED

Malware analysis methods can also be classified according to the techniques used [10] whether we monitor function calls, parameters or if we track the information flow or trace the instructions executed and this is illustrated in Fig. 3 and in Table II you will find a comparison between different supported techniques.

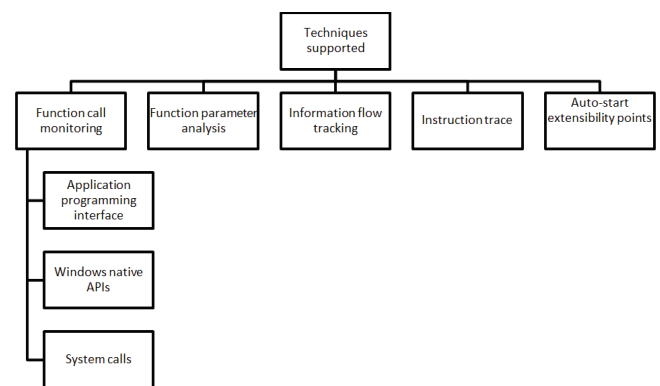


Fig. 3. Classification of malware analysis techniques according to techniques supported.

1) *Function call monitoring*: Function call monitoring can be done by intercepting function calls using hooking by recording the invocation to a log file or analyzing input parameters to monitor the behavior of the program.

Function hooking [30] as shown in Fig. 4 is done by rerouting the function to a different location (customized code) which is the hook function and based on the output the original function sequence continues to execute or it is totally prevented. Hooking function should be implemented carefully as not to be detected by the malware as if this happened malware would change its behavior. Import Address Table (IAT) hooking

Function hooking has two types [19] IAT hooking and inline hooking, IAT is an array of links, which refer to the DLLs imported by the PE loader while inline hooking is done by overwriting the beginning of the target function, by jumping unconditionally to the hook function (trampoline function).

In windows there is Detours library, which is [19] a framework released by Microsoft to place inline hooks. Detours library [21] which can facilitate function hooking by redirecting control flow from the function to hook to the analysis function, also function hooking can be done by debugging techniques and inserting breakpoints or can be done by replacing dynamic shared libraries to perform function call monitoring and this will be discussed later in this section.

Beside function call hooking, to be able to perform function call monitoring we can process function call traces and give richer representations of the malware behavior.

In function hooking Debugging techniques are needed where breakpoints are inserted and when the breakpoint triggers control is given to the debugger and value of memory contents and cpu state can be checked, also function hooking here can be done and whenever a certain message is received (key stroke for example) the hook function is executed.

Concerning replacing dynamic shared libraries it is just another method to implement function hooking where the original libraries are replaced by libraries that contain hooking function.

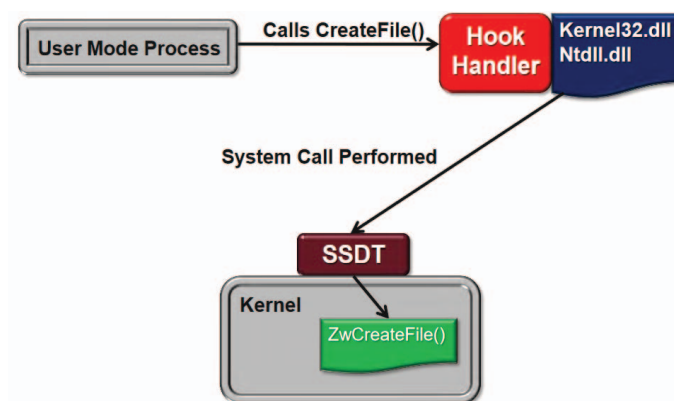


Fig. 4. The hooking process.

These analyzed functions can be:

a) *Application programming interfaces*: APIs are function set that perform set of functionality like networking, system services, security and management, so its important to analyze the APIs to be able to detect if there are any problems or possible malware in the analyzed software.

b) *System calls*: Applications running on your operating system are divided into two major parts either they run in kernel mode or in user mode, normal applications run in user mode while operating system is executed in kernel mode where the code executed in kernel mode has direct access to system state and user mode processes is prevented from this privilege. For example to create a file a user mode application needs to invoke a specific system call to go to kernel mode if the application has a permission to do so.

Therefore its extremely important to check the system calls occurring during the execution of the software under analysis.

c) *Windows native APIs*: Windows native APIs rely in a level between windows API and system calls, where native APIs are called from higher levels to invoke system calls. The problem here is that the normal legal applications communicate with the OS through the windows API while malware may skip this layer and interact with native API directly and sometimes the malware author wants to skip the native windows APIs and call directly from the system calls.

2) *Function parameter analysis*: In function call for example to create a new file the application invokes CreateFile we are interested in the arguments and the return value of the function. Function parameter analysis is done by analyzing and tracking the actual parameters that are passed when the function is invoked as well as tracking the return values from functions and check if they are used in a further function call, all this is needed to analyze the input and output of the function to detect the exact behavior of the malicious software.

To access the function arguments [3] the call back routine has to read it from the virtual system and this can be done by extracting and copying the arguments values whenever the call back routine is invoked and the arguments are put into an array. The issue here is that the number of arguments (size of array) should be determined in run time to be able to copy the right amount of data during call back also its not always possible to read from the virtual address space immediately.

3) *Information flow tracking*: The issue with malware analysis is that sometimes malware has anti-analysis feature where malicious paths are avoided when a malware analysis tool is detected so a solution to execute all program paths is needed. Information flow tracking is concerned with how program processes data and the propagation of interesting data throughout the system while the program using this data is running the data monitored are marked (tainted) and when data are processed the taint label is propagated. This is useful in detecting if the program reveals secret inputs or use controlled inputs unsafely. Information flow tracking is considered from types of dynamic analysis, in this scope you have taint source , which introduce new labels concerning interesting data or input from un-trusted sources and taint sink that reacts in a certain way to tainted input. The tainted input is tracked how it will be propagated.

Aspects needed to be taken into account in information flow tracking are Direct data dependencies, Address dependencies, control flow dependencies and implicit information flow.

In Direct data dependencies, If an operation is directly dependent on a tainted value as shown in Fig. 5

```
//tainted x
a = x;      mov eax, x
a = a + x;  add eax, x
```

Fig. 5. Direct data dependencies.

In Address dependencies the address is dependent on a tainted value either the index of the array or the base of the address. While control flow dependencies based on the tainted value (x) the value of (v) is specified, as shown in Fig. 6

```
//tainted x
if (x == 0) {
    v = 0;
} if (x == 1) {
    v = 1;
} ...
```

Fig. 6. control flow dependencies.

In implicit information flow, the value of the variable v is dependent on the value of x implicitly and here the values of (a) and (b) should be tainted but this type is more difficult to handle. as shown in Fig. 7

```
//tainted x
a = 0; b = 0;
if (x == 1) then a = 1 else b = 1;
if (a == 0) then v = 0;
if (b == 0) then v = 1;
```

Fig. 7. implicit information flow.

Taint sources introduce new labels to the data we are interested in and taint sinks reacts in a specific way as a response of a certain input. In this scope we have direct data dependencies, control flow dependencies or address dependencies.

4) *Instruction trace*: Track and analyze the sequence of machine instructions, which contain very valuable information which is not available on the higher level abstraction. Tracing malicious code at instruction level is not an easy or simple task so in [9] EtherAnnotate which aid in the task of malicious software analysis, it offers an instruction tracer which records the contents of the registers during instruction processing and a graphical annotation plugin.

EtherAnnotate [9] is capable of tracking any instruction trace including that of the operating system itself and values of currently accessed variables are recorded in the instruction trace log where previous analysis techniques do not offer malware analysis in a transparent way.

5) *Auto-start extensibility points (ASEP)*: Auto start Extensibility points (ASEP) [28] refer to part of OS and application extensibility points that can be hooked without direct invocation from the user. Introduce a mechanism that can help applications to start automatically after operating system boots up so this type of points should be monitored when an application from an un-trusted source is executed on the machine. Where a spyware attaches to extensibility points in the system to make sure that the spyware will be started when the system starts where they launch most of the commonly used applications.

ASEPs have can be classified to the following categories [28]:

- ASEPs that start new process
- ASEP that hook system processes
- ASEPs that load drivers
- ASEPs that hook multiple processes
- Application specific ASEPs

There are three methods to monitor an ASEP hooking either to add a new ASEP hook or modify an existing ASEP hook or modify the executable file pointed to by the ASEP.

C. IMPLEMENTATION STRATEGIES

This section is concerned with how running and analyzing the malware sample will be done and the classification of this as shown in Fig. 8 and the pros and cons of each method.

1) *Analysis in user space vs in kernel space*: Analyzing unknown malware sample in user mode gather information about invoked functions or API calls and other high level information but hiding the analysis component when running here is very difficult.

Meanwhile when the analysis tool is running in kernel space more information can be gathered like invoked system calls also if the malware is executing in user space then hiding the analysis tool here is possible, but if the malware granted access in kernel space then the Process of hiding will be again complicated but in this case API hooking can be achieved easily. If the malware analysis is done in kernel mode then methods like information flow tracing and instruction trace can be done so it give more accurate approach than user mode analysis.

2) *Analysis in an emulation environment*: This is done to monitor the running malware and control its effect, different parts of execution environments can be emulated.

From the advantages of emulation [31] is the flexibility of analysis tool development and efficiency for fine grained analysis, which provides insight for malware code layers (polymorphic and metamorphic) polymorphic is when part of the

TABLE II. COMPARISON BETWEEN MALWARE ANALYSIS TECHNIQUES SUPPORTED.

Factors	Function call monitoring	Function parameter analysis	Information flow tracking	Instruction trace	Auto-start extensibility points
Time	Time needed to analyze malware will relatively increase.	More time is taken to get the function arguments.	Dynamic taint analysis may rise an alert too late.	More time is needed for etherAnnotate.	N/A
Input	By injecting a Dynamic Link Library file (DLL) into the targeted process.	Operating system function calls.	Taint source and sink.	Corresponding machine code of the malware source code.	Auto-Start extensibility Points (ASEP) hook functions.
Resource consumption	May cause drop in performance if the targeted hooked function is called many times.	More processing is needed.	Needs much more resources than normal malware analysis methods [22].	More time and storage resources are needed.	N/A
Advantages	API monitoring is very helpful as they it helps to catch any overlooked problem where hooking can trap any function call not just system calls.	More accurate because it does not only checks the functions called but also the arguments and return values.	Taint check is simple and flexible to be used on a variety of programs. It also tracks the flow of interesting data all through program execution. This allows us to detect overwrite attacks and attacks that cause overwriting sensitive values.	More accurate because it is on a low level so it reaches information not available on a higher abstraction level.	Can monitor hooking to ASEPs and based on it either modify or insert a new function.
Limitations	Malware can detect the presence of a hooking function and thus change its behavior.	Takes more time and processing.	Weak in implicit flow handling and also too many dependencies leads to over tainting, which may cause tainting to explode. Also there is the problem of undertainting where dynamic taint analysis does not handle some types of information flow.	It is somehow complicated to trace code activity at the instruction level.	ASEP based approach do not cover programs that auto-start based on non-extensibility methods.

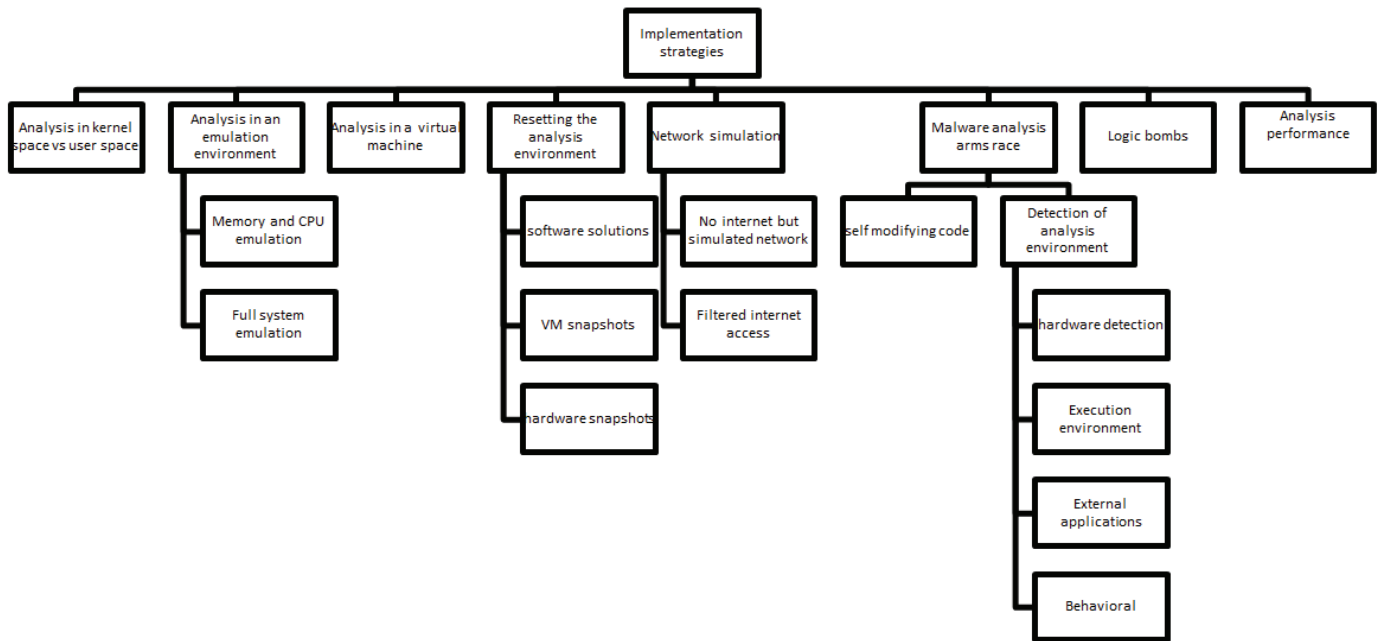


Fig. 8. Classification of malware analysis techniques according to implementation strategies.

code remains constant with each iteration while metamorphic malware is rewritten with each version of the code so its sometimes hard for the antivirus to detect that they are different versions for the same malware, data encryption engine and memory layout.

The problem here is that the malware sample can detect that its running in an emulation environment and stop working or change its behavior and this is called transparency [7], also the problem is whats called semantic gap where the desired information is of higher level like system calls and file handles and this can be inferred from CPU state and memory contents.

The semantic gap is caused because there is a gap between higher level values and those of the guest and this gap has to be bridged.

a) Memory and CPU emulation: This type of emulation allows safe execution of the malware as all side effects the execution produces are contained in a sandbox, modern antivirus programs use this technique. This technique can detect any unpacking sequence and check if the malware sample is packed or obfuscated.

The problem here is that the malicious sample can detect that it is running on an emulator by running an operation that

is not totally emulated so it will be able to detect the presence of the emulator and thus change its behavior.

b) Full system emulation: Act as a real computer system (CPU, memory, peripherals such as network interface cards and mass storage) and here a guest operating system can be installed on the host environment.

The advantage here is that the host and guest architectures can be different. But the problem here is that the malware sample can detect that its running in an emulation environment and stop working or change its behavior, also the problem is whats called semantic gap where the desired information is of higher level like system calls and file handles and this can be inferred from CPU state and memory contents. The semantic gap is caused because there is a gap between higher level values and those of the guest and this gap has to be bridged.

c) Analysis in virtual machine: Virtual machine is the efficient, duplicate, isolated duplicate of a real machine, virtual machine monitor (VMM) assigns hardware to a specific virtual machine here the privilege to the machine is not granted to any virtual machine. Here the architecture of the guest and host should be identical and this is good for improved performance.

In the virtual machine it is not that straightforward to create an instruction trace because unprivileged instructions are executed on bare hardware without including the VMM and then we use kernel space analysis to produce instruction trace.

From the advantages of analysis in virtual machine is that it can monitor multiple machines running on the same real machine, also you can take snapshot of the system before and after running the malware thus the malware behavior can be easily monitored.

The main disadvantage here is the same as that of emulation, which is the loss of semantic information known as the semantic gap problem and also sometimes the malware may detect that it is running on a virtual machine and thus change its behavior. Also sometimes malware may spread to the host and affect it badly.

d) Resetting the analysis environment: The problem here is the time it takes to reset the environment back to the clean state and there are three approaches to do this

- 1) Software solutions:
Take an image of the hard drive that contains the base image and after each analysis the image is restored.
- 2) Virtual machine snapshots:
Its to take a snapshot of the virtual machine state (virtual machine storage, CPU and memory content), this mechanism reduces analysis time as no reboot for the system is required.
- 3) Hardware snapshots:
Save all changes in the image to another physical drive (virtual hard disk) and to reset the system required, it acts as a set of reference makers to the data where the user can roll back to it. The snapshots will be as a tree (parent-child relation) and each snapshot will create a branch of the tree

3) Network simulation:

Any running malware will be in need to network or internet connection and if the internet connection was available for malware this may cause problems and unwanted effects also if the internet connection was totally prevented the behavior of the malware wont be totally monitored. For example if the malware is sending spam files to other devices in the same network and the network connection was unavailable, the malware wont try to send any spam files and the bad behavior wont be detected.

a) No internet but simulated network:

The device isnt connected to the internet but all the internet services are simulated like DNS, email and file transfer services.

b) Filtered internet access:

Here there will be internet access but it will be totally restricted by filtering the outgoing traffic, traffic shaping or rate limiting.

4) Malware analysis arms race:

Arms race is between the malware analysts and the malware authors whereas the malware analysts create better and more advanced tools, the attackers try to make their malware unbreakable by analysts.

a) Self-modifying code and packers:

A large part of malicious code nowadays come in packed format. Malware use self-modifying code to disguise its malicious intents or use packer tools that automatically transform the behavior of the malware binary and create a new executable. The unpacker restore the data into the original representation and the control is handed to the unpacked binary version that perform the intended tasks, but here signature matching can be used to detect that packing was used to create this binary. To decrease the effect of the packing problem we can unpack the binary to detect the effect of the expected malware.

b) Detection of analysis environment:

When the malware detects that its being executed in an analysis environment it either changes its behavior or execute a nonmalicious behavior to deceive the analysis tool. This detection can be done by one of the following:

- 1) Hardware detection:
It is done using the fingerprint of the virtual machines.
- 2) Execution environment:
It checks the effect of a monitored process environment.
- 3) External applications:
Presence of well-known monitoring applications can help Malware detect that there is an analysis system.
- 4) Behavioral:
There is a timing difference between executing on a virtual machine and a real one, if the malware detected this timing difference it will conclude that its running on a virtual machine and change its behavior.

5) Logic bombs:

Malware execute its malicious behavior on a certain date or when a certain key is pressed by the user, malware binary is replaced by an encrypted version and the key will be only available when the code is executed when the trigger occurs.

6) Analysis performance:

Analysis causes noticeable slowdown in the performance especially if extensive analysis is done at the system startup.

To solve this problem we can slow down time in an emulated or a virtual environment but the problem here if the malware reacts with other devices over a network this can cause synchronization problems like timeout or we can give a timeout for the analysis process and expire when the time ends, the latter approach will help avoiding executing the analysis tool all the time on the machine

D. OTHER CLASSIFICATION

1) Memory Analysis:

Malware analysis methods can be classified according to how the tool deals with memory and analyze it where in this category snapshot of physical memory of the computer is analyzed [26] to detect its state.

In-memory data that can be collected can be:

- 1) Processes and threads
- 2) Files handlers
- 3) Network objects
- 4) Modules/Drivers
- 5) Caches
- 6) Windows registry keys

The problem [1] here is that memory analysis has its limitations because memory analysis is a manual process and it needs deep understanding of OS internals to be able to analyze the output, also there will be anti-forensics tools (tools that hide data using techniques like encryption, steganography, artifact wiping, etc), which make memory analysis and data acquisition very hard.

2) Triggers:

Define interesting actions or triggers [1] that when occur its time to take a memory dump.

a) API based:

Check the API call sequences and API Calls and when a malicious behavior is detected a memory dump is taken.

b) Performance based:

Detecting abnormal behavior in the CPU usage and based upon this a memory dump is taken.

c) Instrumentation based:

Detect if there is any abnormal mathematical computation (unpacking, decryption, decoding loops, etc).

V. EXPERIMENT

As mentioned earlier security is a major issue in the scope of IoT and the problem is that using some of conventional analysis tools mentioned in the previous section will be computationally exhausting the IoT device.

One of the most recent proposed solutions was cloud Eyes [25] which is a cloud based malware detection platform mainly dedicated for resource constrained devices like IoT devices, it is signature based approach, which is illustrated before in section IV, it is built on reversible sketch structure

and introduces suspicious bucket cross filtering algorithm, which is the main algorithm used in this system. Cloud Eyes needs less memory cost than other systems like ClamAV [5] and split screen [4] anti virus scanners also Cloud Eyes is more efficient than with respect to time analysis in which the amount of time needed in cloud eyes analysis for malware samples is less than that needed for either ClamAV or split screen scanners. Also cloud eyes has the least number of cache misses as the instruction is fetched. All the previous criteria made cloud Eyes the most suitable software to be used with resource constrained devices like IoT devices as they have limited memory, processing and battery. In cloud eyes performance needs enhancement.

In [2] introduces an embedded security framework considers security issues that affects the IoT devices all through the life cycle of the software development and tries to get the best tradeoff between cost, performance and security and AES-GCM based security protocol that has two components firstly creating capabilities then message encryption and authentication, the advantage here is that it provides security services to resource constrained devices. In this paper also a defensive mechanism against jamming attack is proposed by proposing the threshold jamming countermeasure that has better energy consumption and less delay in time, also this paper proposes a lightweight key management technique (Cluster based mobile key management scheme), which also has less computational overhead and less energy consumption and less delay in time than Energy-Efficient Distributed Deterministic Key (EDDK) Management technique, In this reserach the efficient can be improved by using lightweight cryptography and secure Operating system and secure storage, also jamming attack only is considered but no other attacks types in IoT are put into consideration and we need to consider privacy, authentication, authorization at the same time.

In [24] a system that proposes an in-Hub security manager for vulnerable IoT devices, which intercepts all in and out traffic to and from smart home devices to increase security for these devices.

In [20] a secure compiler for developing secure IoT services is proposed as secure software is needed in IoT devices to avoid software vulnerabilities and this is done by using a compiler and virtual machine that use secure coding rules and checking tools. This approach solve the malicious software problem in two stages prevention stage in which secure compiler removes security weaknesses during writing the code and monitoring stage in which the virtual machine monitor any unwanted malicious behavior by the software on the IoT device like buffer overflow attack. This proposed compiler checks more code weaknesses than previous compilers like Fortify and the proposed Virtual machine has better results in detecting buffer overflow attacks but causes more overhead. The problem is that adding the analysis modules to compilers need to be automated, also better performance and accuracy is needed by compaction of Virtual machine.

VI. CONCLUSION AND FUTURE WORK

This paper discussed security and privacy issues in IoT devices and why security in IoT is a major issue and currently under research. Then some of the systems that tried to propose

solutions in this scope, also this paper discussed the taxonomy of the malware analysis tools and the pros and cons of each method.

As a general conclusion IoT devices have restricted resources (memory, time and computation) so they need an optimized malware analyzer that can detect malware attacks and prevent its effect. The problem is that most of current malware analysis tools have very high computational cost so there were many trials for systems that propose solutions for this issue in the IoT scope but still it needs enhancement either in accuracy or performance. As a future work systems that are more optimized in memory and time are needed to be used in IoT devices.

REFERENCES

- [1] *Anti-Forensics Anti-Forensics (PDF Download Available)*. Nov. 2017. URL: https://www.researchgate.net/publication/268290676_Anti-Forensics_Anti-Forensics.
- [2] Sachin D Babar. "Security framework and jamming detection for internet of things". In: *Videnbasen for Aalborg UniversitetVBN, Aalborg UniversitetAalborg University, Det Teknisk-Naturvidenskabelige Fakultet-The Faculty of Engineering and Science* (2015).
- [3] Ulrich Bayer et al. "Dynamic analysis of malicious code". In: *Journal in Computer Virology* 2.1 (2006), pp. 67–77.
- [4] Sang Kil Cha et al. "SplitScreen: Enabling efficient, distributed malware detection". In: *Journal of Communications and Networks* 13.2 (2011), pp. 187–200.
- [5] cisco. *ClamAV*. URL: <https://www.clamav.net>.
- [6] Swagatam Das et al. *Proceedings of the 4th International Conference on Frontiers in Intelligent Computing: Theory and Applications (FICTA) 2015*. Vol. 404. Springer, 2015.
- [7] *Debugging Basics: Breakpoints*. URL: [https://msdn.microsoft.com/en-us/library/4607yxb0\(v=vs.100\).aspx](https://msdn.microsoft.com/en-us/library/4607yxb0(v=vs.100).aspx).
- [8] Tim Dierks and Christopher Allen. "The TLS protocol version 1.0". In: (1999).
- [9] Joshua Michael Eads. "EtherAnnotate: a transparent malware analysis tool for integrating dynamic and static examination". In: (2010).
- [10] Manuel Egele et al. "A survey on automated dynamic malware-analysis techniques and tools". In: *ACM computing surveys (CSUR)* 44.2 (2012), p. 6.
- [11] Mahmoud Elkhodr, Seyed Shahrestani, and Hon Cheung. "The internet of things: new interoperability, management and security challenges". In: *arXiv preprint arXiv:1604.04824* (2016).
- [12] Fireeye. *ApateDNS*. Version 1.0. URL: <https://www.fireeye.com/services/freeware/apatedns.html>.
- [13] Savan Gadhiya and Kaushal Bhavsar. "Techniques for malware analysis". In: *International Journal of Advanced Research in Computer Science & Software Engineering (IJARCSSE)* 3.4 (2013).
- [14] Gerald Combs. *Wireshark . Go Deep*. Version 2.4.2. URL: <https://www.wireshark.org/>.
- [15] Hobbit. *Ncat - Netcat for the 21st century*. Version 7.60. URL: <https://nmap.org/ncat/>.
- [16] *Internet of Things (IoT)*. Oct. 2017. URL: <https://www.cisco.com/c/en/us/solutions/internet-of-things/overview.html#~:stickynav=1>.
- [17] Kris Kendall and Chad McMillan. "Practical malware analysis". In: *Black Hat Conference, USA*. 2007, p. 10.
- [18] Christopher Kruegel. "Full system emulation: Achieving successful automated dynamic analysis of evasive malware". In: *Proc. BlackHat USA Security Conference*. 2014.
- [19] Ajay Kumar and Shubham Goyal. "Advance Dynamic Malware Analysis Using Api Hooking". In: ().
- [20] YangSun Lee, Junho Jeong, and Yunsik Son. "Design and implementation of the secure compiler and virtual machine for developing secure IoT services". In: *Future Generation Computer Systems* 76 (2017), pp. 350–357.
- [21] Microsoft. *Detours Express*. Version v3.0 Build 343. URL: <https://www.microsoft.com/en-us/download/details.aspx?id=52586>.
- [22] James Newsome and Dawn Song. "Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software". In: (2005).
- [23] Ahmad-Reza Sadeghi, Christian Wachsmann, and Michael Waidner. "Security and privacy challenges in industrial internet of things". In: *Design Automation Conference (DAC), 2015 52nd ACM/EDAC/IEEE*. IEEE. 2015, pp. 1–6.
- [24] Anna Kornfeld Simpson, Franziska Roesner, and Tadayoshi Kohno. "Securing vulnerable home iot devices with an in-hub security manager". In: *Pervasive Computing and Communications Workshops (PerCom Workshops), 2017 IEEE International Conference on*. IEEE. 2017, pp. 551–556.
- [25] Hao Sun et al. "CloudEyes: Cloud-based malware detection with reversible sketch for resource-constrained internet of things (IoT) devices". In: *Software: Practice and Experience* 47.3 (2017), pp. 421–441.
- [26] Tomer Teller and Adi Hayon. "Enhancing automated malware analysis machines with memory analysis". In: *Black Hat USA* (2014).
- [27] Dolly Uppal, Vishakha Mehra, and Vinod Verma. "Basic survey on malware analysis, tools and techniques". In: *International Journal on Computational Sciences and Applications (IJCSA)* 4.1 (2014), pp. 103–12.
- [28] Yi-Min Wang et al. "Gatekeeper: Monitoring Auto-Start Extensibility Points (ASEPs) for Spyware Management." In: *LISA*. Vol. 4. 2004, pp. 33–46.
- [29] *Welcome to the INetSim project homepage!* URL: <http://www.inetsim.org/>.
- [30] Carsten Willems, Thorsten Holz, and Felix Freiling. "Toward automated dynamic malware analysis using cwsandbox". In: *IEEE Security & Privacy* 5.2 (2007).
- [31] Lok Kwong Yan. "Transparent and precise malware analysis using virtualization: from theory to practice". PhD thesis. Syracuse University, 2013.